AccessOps

IT Access Management Dashboard

Cotiviti IT Engineering | Internal Tool

Document prepared: July 02, 2026

Classification: Internal / Confidential

Project Summary

AccessOps is a web-based Identity & Access Management (IAM) dashboard built for Cotiviti IT administrators. It provides real-time visibility into user access across Okta SSO and Active Directory, with built-in approval workflows, tamper-evident audit logging, bulk offboarding, and a Ticket Distribution module for shift-based helpdesk operations.

The system integrates directly with the Okta REST API and Active Directory via LDAP, using API keys and service account credentials managed securely through environment variables. All credentials are loaded at runtime and never hardcoded into source files.

TECH STACK	Python 3.14 + Flask 3.0	Vanilla HTML/CSS/JS	Okta API v1	LDAP3 (NTLM)
PORT	5050 (configurable via FLASK_PORT)
DATA STORAGE	Append-only JSONL files (audit_log, access_requests, offboarded_users)
AUTHENTICATION	Okta SSWS token + AD NTLM service account (both via .env)
FILE	it-access-dashboard.html + main.py + .env

How API Keys Are Used

AccessOps relies on two sets of credentials to connect to external systems: an Okta API token for SSO and identity management, and an Active Directory service account for LDAP operations. Neither credential is ever hardcoded. Both are loaded exclusively from environment variables at server startup via python-dotenv.

1. The .env File - Where Credentials Live

All credentials are stored in a single .env file in the project root. This file is never committed to source control.

```
# Okta
OKTA_DOMAIN=cotiviti.okta.com
OKTA_API_TOKEN=ssws_00xxxxxxxxxxxxxxxxxxxxxxxxxx

# Active Directory
AD_SERVER=ldap://ad.cotiviti.com
AD_DOMAIN=cotiviti.com
AD_BIND_USER=svc-accessops@cotiviti.com
AD_BIND_PASSWORD=--------------

# App config
FLASK_PORT=5050
FLASK_DEBUG=false
```

2. Loading Credentials at Startup (main.py)

When main.py starts, python-dotenv reads the .env file and injects all variables into the process environment. The app then reads each variable using os.environ.get():

```
  load_dotenv()                                    # reads .env into os.environ

OKTA_DOMAIN    = os.environ.get('OKTA_DOMAIN', '')    # e.g. cotiviti.okta.com
OKTA_API_TOKEN = os.environ.get('OKTA_API_TOKEN', '') # SSWS token
AD_SERVER      = os.environ.get('AD_SERVER', '')      # ldap://...
AD_BIND_USER   = os.environ.get('AD_BIND_USER', '')   # service account UPN
AD_BIND_PASS   = os.environ.get('AD_BIND_PASSWORD', '')
```

If any required credential is missing the app logs a warning on startup but continues running. The first API call that needs the missing credential will return a 500 error with a clear message - it never silently fails.

3. Okta API Token - How It Is Used

Every call to the Okta REST API attaches the token in an Authorization header using Okta's SSWS (Session Web Service) scheme. A helper function builds this header once and reuses it:

```
  def _okta_headers() -> dict:
    if not OKTA_API_TOKEN:
        raise RuntimeError('OKTA_API_TOKEN not set in environment.')
    return {
        'Authorization': f'SSWS {OKTA_API_TOKEN}',
        'Accept':        'application/json',
        'Content-Type':  'application/json',
    }
```

Okta Action	HTTP Method	Endpoint Called	Token Required
Look up user by email	GET	/api/v1/users^filter=profile.email eq ...	Yes

Get user groups	GET	/api/v1/users/{id}/groups	Yes
Get app assignments	GET	/api/v1/users/{id}/appLinks	Yes
Suspend account	POST	/api/v1/users/{id}/lifecycle/suspend	Yes
Reactivate account	POST	/api/v1/users/{id}/lifecycle/activate	Yes
Reset MFA factors	POST	/api/v1/users/{id}/lifecycle/reset_factors	Yes
Reset password	POST	/api/v1/users/{id}/lifecycle/reset_password	Yes
Remove app assignment	DELETE	/api/v1/apps/{app_id}/users/{user_id}	Yes

API Keys & Credentials - Continued

4. Active Directory Service Account

Active Directory is accessed via LDAP using the ldap3 library with NTLM authentication. The service account credentials (AD_BIND_USER and AD_BIND_PASSWORD) are passed directly from the environment to the ldap3 Connection object:

```
def _ad_connect():
  from ldap3 import Server, Connection, ALL, NTLM
  server = Server(AD_SERVER, get_info=ALL)
  conn   = Connection(
      server,
      user=AD_BIND_USER,        # loaded from AD_BIND_USER in .env
      password=AD_BIND_PASS,    # loaded from AD_BIND_PASSWORD in .env
      authentication=NTLM,
      auto_bind=True
  )
  return conn
```

AD Operation	LDAP Action	Credential Used
Look up user by email	SEARCH (mail=email)	AD_BIND_USER + AD_BIND_PASSWORD
Disable AD account	MODIFY userAccountControl (bit 0x2)	AD_BIND_USER + AD_BIND_PASSWORD
Remove from security group	MODIFY group.member (MODIFY_DELETE)	AD_BIND_USER + AD_BIND_PASSWORD

5. End-to-End Credential Flow

```
.env file
    -
    -  python-dotenv (load_dotenv)
    -
os.environ  --- OKTA_API_TOKEN  ---  _okta_headers()  ---  Authorization: SSWS <token>
            --- AD_BIND_USER    ---  _ad_connect()    ---  NTLM bind to AD server
            --- AD_BIND_PASS    ---  _ad_connect()    ---  NTLM bind to AD server
    -
    -  Credentials never reach the browser
    -  API responses contain user data only - no tokens, no passwords
    -
Frontend (it-access-dashboard.html)  ---  JSON response  --  Flask route
```

6. Security Design for API Keys

Security Control	How It Is Applied
No hardcoded secrets	All tokens/passwords read from os.environ - zero credentials in source code
Token never sent to browser	Flask routes return JSON data only - the OKTA_API_TOKEN is server-side only
Credential validation	Startup logs warn if OKTA_DOMAIN or OKTA_API_TOKEN is missing
Scoped Okta token	Token should have read + lifecycle scopes only - not super-admin
Minimal AD permissions	Service account has read + targeted modify only - not domain admin
SSWS auth scheme	Okta SSWS tokens are revocable and auditable in the Okta Admin Console
.env not committed	.env listed in .gitignore - .env.example provided as safe template

NTLM over LDAP	AD password never transmitted in plaintext - NTLM challenge-response

! Important: Rotating API Keys

If the OKTA_API_TOKEN is rotated or an AD service account password changes, update the .env file and restart the Flask server (python main.py). No code changes are needed - the new value is picked up automatically on next startup via load_dotenv().

Features & Architecture

Module Overview

Module	What It Does	Uses API Key^
Users	Search users, view Okta profile, AD account, app access, licenses	Yes - Okta + AD
Applications	All application assignments across the org	Yes - Okta
Licenses	Microsoft 365 and software license management	Yes - Okta
Requests	Approve or deny access requests	Yes - Okta
Audit Log	Tamper-evident log of every IAM action	Stored locally
Agent View	Side-by-side user comparison with risk flags	Yes - Okta + AD
Bulk Offboarding	CSV upload - review - execute for multiple users	Yes - Okta + AD
Ticket Distribution	Assign helpdesk tickets to agents by shift and specialization	Client-side only

System Architecture

```
Browser  (it-access-dashboard.html)
    -
    -   REST API calls  fetch('/api/...')
    -
Flask Backend  (main.py)  :5050
    -
    --- Okta REST API  (HTTPS + SSWS token)  --  cotiviti.okta.com
    --- Active Directory  (LDAP/NTLM)         --  ad.cotiviti.com
    --- Local data/  (JSONL files)
            --- audit_log.jsonl
            --- access_requests.jsonl
            --- offboarded_users.jsonl
```

Project Files

File	Extension	Purpose
it-access-dashboard	.html	Single-page frontend - all UI, CSS and client-side JS
main	.py	Flask REST API server - all backend logic and integrations
README	.md	Setup instructions, API endpoints, CSV format reference
ARCHITECTURE	.md	System design, component map, data flow diagrams
requirements	.txt	Pinned Python package versions for reproducible installs
requirements	.in	Source dependency list (used to regenerate requirements.txt)
.env	.env	Runtime credentials - OKTA_API_TOKEN, AD_BIND_PASSWORD, etc.

Setup & API Reference

Getting Started

1	**Create virtual environment**	`uv venv .venv`
2	**Activate venv**	`.venv\Scripts\Activate.ps1`
3	**Install dependencies**	`uv pip install -r requirements.txt`
4	**Configure credentials**	`Edit .env - add OKTA_API_TOKEN, AD_BIND_USER, AD_BIND_PASSWORD`
5	**Start the server**	`python main.py`
6	**Open in browser**	`http://localhost:5050`

API Endpoints

Method	Endpoint	Description	Auth
GET	/api/okta/user^email=	Fetch Okta profile + groups + apps	SSWS token
GET	/api/ad/user^email=	Fetch AD account + group memberships	NTLM
POST	/api/okta/user/{id}/suspend	Suspend Okta account	SSWS token
POST	/api/okta/user/{id}/activate	Reactivate Okta account	SSWS token
POST	/api/okta/user/{id}/reset-mfa	Clear all MFA factors	SSWS token
POST	/api/okta/user/{id}/reset-password	Send password reset email	SSWS token
POST	/api/okta/user/{id}/remove-app	Remove app assignment	SSWS token
POST	/api/ad/user/disable	Disable AD account (UAC bit 0x2)	NTLM
POST	/api/ad/user/remove-group	Remove user from AD security group	NTLM
GET	/api/requests	List all access requests	None
POST	/api/requests	Submit a new access request	None
POST	/api/requests/{id}/approve	Approve request	None
POST	/api/requests/{id}/deny	Deny request	None
POST	/api/offboard/preview	Parse CSV, return preview	SSWS + NTLM
POST	/api/offboard/execute	Execute bulk offboarding	SSWS + NTLM
GET	/api/offboard/history	View past offboarding records	None
GET	/api/audit	Retrieve last 200 audit entries	None
GET	/api/health	Okta + AD connectivity status	SSWS + NTLM

Production Hardening Checklist

- Run behind nginx reverse proxy with HTTPS/TLS
- Add authentication to Flask app (Okta SSO or Basic Auth) - currently open internally
- Restrict CORS from wildcard (*) to your internal domain only
- Move JSONL data files to PostgreSQL or SQLite for reliability
- Add flask-limiter rate limiting to all POST endpoints
- Use a dedicated Okta API token with minimum required scopes (not super-admin)
- Use an AD service account with read + targeted modify permissions only
- Store .env secrets in a secrets manager (Vault, AWS Secrets Manager) in production
- Rotate OKTA_API_TOKEN and AD_BIND_PASSWORD on a regular schedule

This document was auto-generated from the AccessOps source files. For the latest information refer to README.md and

ARCHITECTURE.md in the project folder.